PrivyLens

Viva guide: what every folder is, how each part was built from scratch, and the answers you will need.

> Send the report to [redacted] at [redacted].
>
> `Send the report to [PERSON_1] at [EMAIL_1].`

That is the whole project: private details are replaced by labelled placeholders on your own computer, before the text reaches ChatGPT, Claude or any website. The real values stay in a local "vault", so you can swap them back into the reply.

Read order for a one-evening cram: sections 1, 2, 3, then 9 (questions), then 8 (weak spots). Sections 4–7 are for when the examiner digs.

Contents

1. The 60-second pitch
2. Words you must know (plain English)
3. How the system works end to end
4. Folder and file map
5. How we built each part from scratch
6. The results, and how to read them
7. Why these design choices
8. Weak spots: say them before they do
9. Likely viva questions with answers
10. Live demo script
11. Numbers to memorise

1. The 60-second pitch

People paste emails, chat logs and documents into AI tools without noticing the names, phone numbers and ID numbers inside. Once sent, it is out of their control.

> **Say this (out loud, twice, before the viva)**
> PrivyLens finds personal information in text and replaces it with typed placeholders like [PERSON_1] and [EMAIL_1] *on the user's own computer*, before the text is shared. The real values stay in a local vault, so the AI's answer can be restored with the real names. The detector is a small Transformer that we wrote ourselves in PyTorch, with our own tokenizer, trained from scratch. We compared it against a regex baseline, a from-scratch encoder-decoder, and a fine-tuned DistilBERT. Our best system is a hybrid: regex for structured data like emails and PAN, plus our Transformer for names, places and organisations. It has 1.56 million parameters and a 5.5 MB model file, so it can run locally. It is shipped as a Chrome extension.

The four things the project proves

1. **A model built from scratch works.** 1.56M parameters reaches F1 0.92 on the standard test set.
2. **Tagging beats generating for redaction.** Our encoder-decoder rewrote ordinary words in 35% of test rows. A tagger cannot change the text.
3. **Regex plus a model beats either alone.** Regex has 0.00 F1 on names; the model is weaker than regex on emails.
4. **Small and local is possible.** The pretrained DistilBERT is better (F1 0.956) but 40 times bigger, 65.2M parameters and about 261 MB.

2. Words you must know

You do not need the maths. You need to say each of these in one sentence without hesitating.

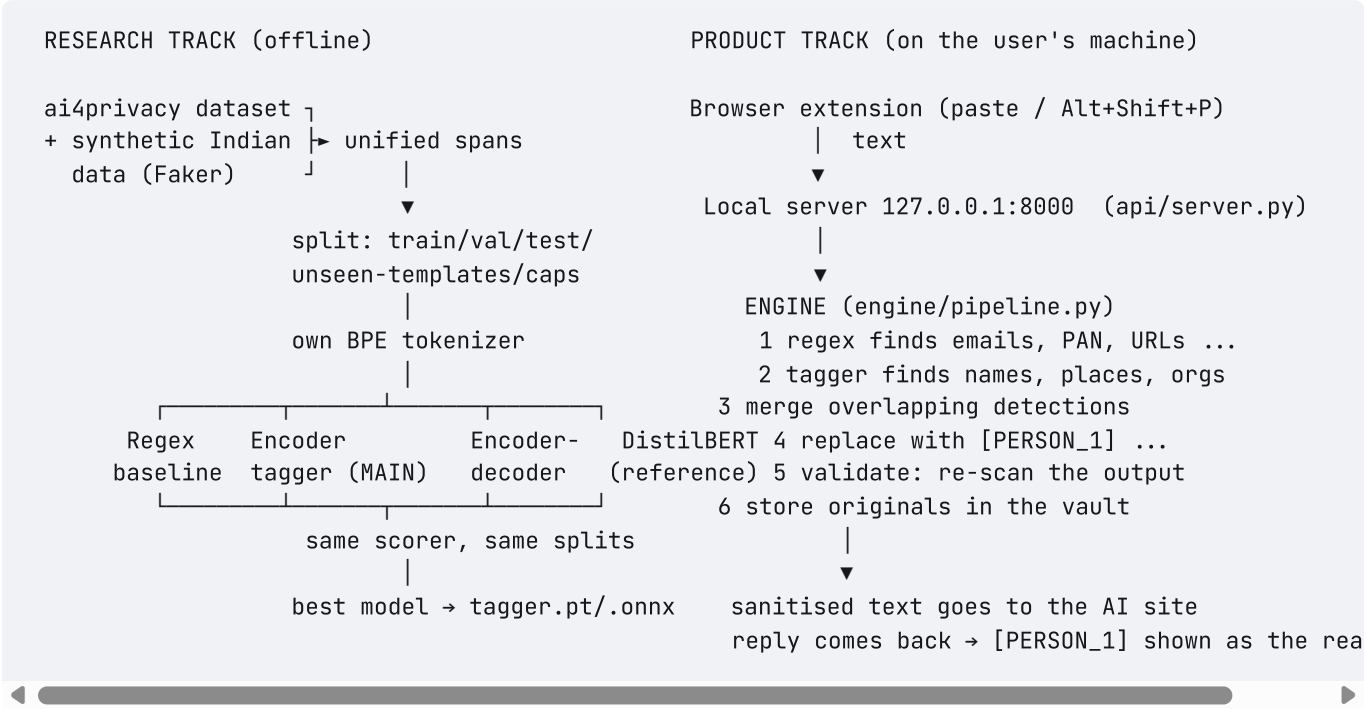
Term	Plain-English meaning, and where it is in our project
PII	Personally identifiable information: anything that points to a real person (name, email, phone, ID number, address).
NLP	Natural language processing: making computers work with human text. Ours: finding private details in sentences.
NER / sequence labelling	Named entity recognition. Give every word a label, such as "this word is part of a person's name". Our model does this.
BIO tags	The labelling scheme. `B-PERSON` = first word of a name, `I-PERSON` = continues it, `O` = not private. We have 11 entity types, so 1 + 2×11 = **23 tags**.
Span	A stretch of text with a start position, end position and label. Example: characters 19–31 are a PERSON.
Token / tokenizer	A token is a piece of text the model reads (a word or part of a word). A tokenizer cuts text into tokens and turns them into numbers. Ours is the BPE tokenizer in `tokenizer/bpe.py`.
BPE (byte-pair encoding)	Start with single characters. Repeatedly merge the most frequent neighbouring pair into one new token, until the vocabulary is full (8,000). Common words become one token; rare strings such as IDs split into pieces. We wrote it ourselves.
Embedding	Each token number is looked up in a table and becomes a list of 128 numbers (a vector). Words with similar roles end up with similar vectors.
Positional encoding	Attention alone cannot tell word order, so we add a position signal to each vector. Ours uses fixed sine and cosine waves (the original Transformer paper's way).
Self-attention	Every word looks at every other word and decides how much each matters, then mixes in information from the useful ones. This is how "Rahul" gets context from "to" and "Sharma".
Multi-head attention	Run attention several times in parallel (4 heads), each with its own focus, then combine. Like four people reading the same sentence for different clues.
Feed-forward network	A small two-layer network applied to each word separately after attention. Extra thinking capacity.
Residual connection + LayerNorm	"Add the input back to the output" so information is not lost, then normalise the numbers so training stays stable.
Encoder / decoder	Encoder reads the whole input at once. Decoder writes output one token at a time. Our tagger is encoder-only; the comparison model has both.
Causal mask / padding mask	Causal: the decoder may not peek at future words. Padding: attention ignores filler tokens added to make a batch equal length.
Teacher forcing	While training the decoder, feed it the *correct* previous words instead of its own guesses, so it learns faster.
Cross-entropy loss	The score of how wrong the model is: it punishes confident wrong answers. Training means making this number fall.

AdamW, learning rate, warm-up, cosine decay	AdamW is the optimiser that adjusts the weights. Learning rate is the step size (we use 5e-4). Warm-up starts small for the first 5% of steps; cosine decay then shrinks it smoothly to near zero.
Epoch / batch	An epoch is one full pass over the training data. A batch is a group of 32 examples processed together.
Overfitting	The model memorises training data and does worse on new data. Our validation score flattens near epoch 20 while training loss keeps falling: mild overfitting.
Train / val / test	Train: learn from it. Val: pick the best checkpoint. Test: final exam, touched only for reporting.
Precision / recall / F1	Precision: of the things we hid, how many were truly private. Recall: of the truly private things, how many we hid. F1: one number combining both. For privacy, **recall matters most** (a miss is a leak).
Fine-tuning	Taking a model already pretrained by others on huge text, and training it a little more on our task. We did this only with DistilBERT, as a reference.
Parameters	The adjustable numbers inside a model. More parameters = bigger, slower, more capable. Ours: 1.56M. DistilBERT: 65.2M.
ONNX	A portable model file format that runs without PyTorch (even in a browser). We export our tagger to it (5.5 MB).
Ablation	Change exactly one thing (layers, heads, vocab size) and measure what happens, to learn what matters.
Hybrid engine	Regex + neural model combined, with a rule for who wins when they overlap.
Vault	A table kept in memory on the user's machine: `[PERSON_1]` → "Rahul Sharma". Used to restore replies.

3. How the system works end to end

Two tracks

Research track (done once, mostly on Google Colab GPU): build data, tokenizer, models; train; compare.
Product track (runs every time a user pastes text): the engine, the local API and the browser extension.



What happens when a user pastes a sentence

1. **Extension catches the paste** in `content.js` and passes it to `background.js`, which is allowed to call the local server.
2. **Server** `api/server.py` receives it at `POST /sanitise` (only reachable from this computer).
3. **Regex** (`baselines/regex_detector.py`) finds structured items with patterns.
4. **Tagger** (`models/encoder_tagger.py`): the BPE tokenizer cuts the text into tokens and records where each token sits in the original characters. The Transformer gives each token a tag. Tags are joined back into character spans. Long text is cut into overlapping 128-token windows and the probabilities are averaged.
5. **Word snapping** (`snap` in pipeline.py) widens each model span to whole words, so "MIH" inside "MIHIR" can never be left half visible.
6. **Merge** chooses one winner when spans overlap: precise regex (email, URL, IP, PAN) beats the model, which beats loose regex (phone, Aadhaar, date, ID).
7. **Redact**: each value becomes a numbered placeholder; the same value always gets the same number.
8. **Validate**: the engine re-scans its own output with regex. Any structured PII still visible blocks the result. Spans where the model confidence is below 0.8 are flagged for the user.

9. **Vault** remembers placeholder → real value for the session. `POST /restore` swaps them back into the AI's reply.

4. Folder and file map

Everything under `privylens/` is the Python package. Line counts are real: the whole project is about 2,000 lines, deliberately minimal.

```
Browser Privacy Agent/
├── PrivyLens_Project_Spec.md   the original plan (10 weeks, 4 models + engine + extension)
├── CHECKLIST.md                what was built, ticked off week by week
├── requirements.txt            torch, numpy, faker, datasets, scikit-learn, matplotlib, pytest,
│                               transformers (DistilBERT only), fastapi, uvicorn, onnx, onnxruntime
├── privylens/                  THE PYTHON PACKAGE
│   ├── schema.py               the 11 entity types and the 23 BIO tags (shared by everything)
│   ├── data/
│   │   ├── synthetic.py        fake Indian data: sentence templates filled with Faker values
│   │   ├── loaders.py          converts public ai4privacy dataset to our format, merges with synthetic
│   │   ├── splitter.py         train / val / test / unseen_templates / test_caps + case augmentation
│   │   └── seq2seq_pairs.py    builds (original → sanitised) pairs for the encoder-decoder
│   ├── tokenizer/
│   │   ├── pretokenize.py      first cut into words; keeps  @ . -  inside so emails stay whole
│   │   ├── bpe.py              our own BPE trainer + encoder + decoder
│   │   └── align.py            character spans ⇄ per-token BIO tags
│   ├── models/
│   │   ├── layers.py           the Transformer parts from scratch: embedding, positions,
│   │   │                       multi-head attention, feed-forward, encoder block, decoder block, masks
│   │   ├── encoder_tagger.py   MAIN MODEL (encoder stack + linear head) and predict()
│   │   └── seq2seq.py          encoder-decoder with teacher forcing and greedy decoding
│   ├── baselines/regex_detector.py   hand-written patterns (the lower bound)
│   ├── training/
│   │   ├── train_tagger.py     loop: AdamW, warm-up + cosine, saves best checkpoint
│   │   ├── train_seq2seq.py    same idea for the encoder-decoder
│   │   └── train_distilbert.py reference model; the ONLY file that imports Hugging Face
│   ├── evaluation/
│   │   ├── metrics.py          span P/R/F1 per entity, leaked spans, leaked chars, over-redaction
│   │   ├── preservation.py     scoring for generated text (hallucinated / dropped / changed)
│   │   └── model_summary.py    draws the architecture figure with real parameter counts
│   ├── engine/
│   │   ├── pipeline.py         the hybrid engine: detect, snap, merge, redact, validate, vault, restore
│   │   └── export_onnx.py      tagger.pt → tagger.onnx and checks the outputs match
│   └── docs/pdf.py             redact whole PDFs (true removal of text, not just black boxes)
├── api/server.py               FastAPI: /sanitise and /restore, localhost only
├── extension/                  Chrome extension, Manifest V3, plain JavaScript
│   ├── manifest.json           permissions, supported chat sites, Alt+Shift+P shortcut
│   ├── background.js           makes the call to the local server on behalf of web pages
│   ├── content.js              runs inside the page: cleans pastes, restores replies, preview
│   ├── shared.js               helpers shared by popup and page (highlight colours, offsets)
│   └── popup.html / .css / .js the toolbar popup: manual mode, "protect this site", settings
├── tests/                      pytest: attention, bpe, align, regex, tagger, seq2seq, engine, api
├── tokenizers/                 trained BPE files: 4k, 8k, 16k, and bpe_8k_caps.json (production)
├── notebooks/colab_setup.ipynb clones the repo into Colab, installs, mounts Drive, checks the GPU
├── reports/
│   ├── results.md              every table, ablation and failure analysis
│   └── figures/tagger_architecture.png
├── data/        (git-ignored)  raw / processed / splits / synthetic  (rebuilt by scripts)
├── checkpoints/ (git-ignored)  tagger.pt (6.3 MB), tagger.onnx (5.5 MB), earlier runs
└── pdf/ , pdf_out/ (ignored)   sample PDFs in, redacted PDFs + reports + vaults out
```

> **Why some spec files are missing**
>
> The spec planned many small files (embeddings.py, attention.py, merger.py, vault.py ...). We merged them into fewer files (layers.py, pipeline.py) to keep the code minimal. If asked, say: same components, fewer files, by choice.

Why data and checkpoints are not in git

The ai4privacy dataset licence is academic use only and forbids redistribution, so data is never committed. Scripts rebuild it. Checkpoints are large binaries.

5. How we built each part from scratch

"From scratch" means: no pretrained weights, and no ready-made Transformer classes. We use PyTorch only for basic pieces (Linear, LayerNorm, Embedding, Dropout, optimiser) and wrote the Transformer logic ourselves.

5.1 Schema `schema.py`

11 entity types in two groups. **Structured** (patterns): EMAIL, PHONE, PAN, AADHAAR, URL, IP_ADDRESS, DATE, ID_NUMBER. **Contextual** (need understanding): PERSON, ORGANIZATION, LOCATION. This split drives the whole hybrid design.

5.2 Data `data/`

1. **Public data.** ai4privacy PII dataset (English). `loaders.py` maps its 28 labels onto our 11 (for example GIVENNAME and LASTNAME → PERSON; CITY, STATE, STREET → LOCATION) and cuts long documents into short chunks.
2. **Synthetic Indian data.** `synthetic.py` has sentence templates like "Send it to {PERSON} at {EMAIL}" and fills the slots using Faker's `en_IN` locale, with valid-format PAN, Aadhaar-style numbers and Indian phone numbers. While filling, it records the exact character start/end of each value. That gives free, perfect labels. Every value is fake: no real person's data. 5,000 rows.
3. **One format.** Every example is `{text, spans:[{start,end,label}]}`. Character spans are the single source of truth; BIO tags and seq2seq targets are both derived from them.
4. **Splits** (`splitter.py`): train 39,806 · val 3,956 · test 3,993 · **test_unseen_templates** 595 · **test_caps** 1,997. Split by document id so pieces of one document never leak across splits. A few synthetic templates are removed from everything else to form the unseen set: it measures whether the model learned the idea or only memorised sentence shapes.
5. **Case augmentation.** A user typed an ALL-CAPS name and it was missed: only 41 of 17,772 PERSON training examples were upper case. We added random UPPER (15%) and lower (10%) case copies to *train only*, retrained the tokenizer, and made the `test_caps` set. Val, test and unseen stayed identical, so old and new models remain comparable.

5.3 Tokenizer `tokenizer/`

1. **Pre-tokenise** (`pretokenize.py`): a regex cuts text into words but keeps `@ . -` inside a word, so `rahul@example.com` is one word, while a trailing full stop is split off. A word after a space gets a "_" marker so decoding can restore spaces.
2. **Train BPE** (`bpe.py`): start from single characters, count adjacent pairs, merge the most frequent, repeat until 8,000 tokens (or no pair occurs twice). Trained on the **train split only**, so the tokenizer never sees test text. It stores merges in learned order and applies the earliest-learned merge first when encoding.
3. **Offsets.** Every token remembers its (start, end) in the original text. That is how a prediction on tokens becomes a redaction of exact characters.

4. **Align** (`align.py`): `to_bio` converts gold spans to per-token tags for training (first token `B-`, the rest `I-`). `to_spans` converts predicted tags back into character spans. A self-check shows how many gold spans are recoverable from tokens (the ceiling for any model).

5. Tokenizer variants of 4k, 8k and 16k vocabulary were trained for the ablation. Production uses `bpe_8k_caps.json`.

5.4 The Transformer parts `models/layers.py` (127 lines)

Piece	What our code does
Embedding	Table lookup then multiply by $\sqrt{d}$ (d = 128). Initialised with small values (std $d^{-0.5}$). A bug here is in our history: PyTorch's default made tokens ~11× louder than the position signal and attention saturated, so the first run scored F1 0.21 until fixed.
Positional encoding	sin/cos of different frequencies, registered as a fixed buffer. A learned version exists for the ablation.
Multi-head attention	Linear maps make Q, K, V. Split into 4 heads (32 numbers each). scores = $Q \cdot K^T / \sqrt{32}$. Blocked positions get a huge negative number before softmax, so they become weight 0. Weights multiply V; heads are merged and passed through an output Linear. Same class does cross-attention when given the encoder output as K and V. It keeps its last weights for heat-maps.
Feed-forward	Linear 128→256, GELU, dropout, Linear 256→128.
Encoder block	x = LayerNorm(x + attention(x)); then x = LayerNorm(x + feedforward(x)).
Decoder block	Masked self-attention, then cross-attention over the encoder, then feed-forward; each with residual + LayerNorm.
Masks	Padding mask and causal (lower-triangular) mask.

Tested by `tests/test_attention.py` (shapes and masking correctness).

5.5 The main model `encoder_tagger.py`

```
token ids → Embedding(128) × √128 → + sinusoidal positions → dropout
          → 4 × [ multi-head attention (4 heads) → Add&Norm → feed-forward (128→256→128) → Add&Nor
          → Linear(128 → 23 tags)  → one score per tag per token
```

Where the 1.56M parameters live	Count
Embedding table 8,000 × 128	1,024,000
4 encoder blocks (≈132k each)	529,920
Output layer 128 × 23 + bias	2,967
Total	**≈1,556,887**

Two thirds of the model is the vocabulary table. That is why the 4k vocabulary needs only 0.78M parameters.

5.6 Training `train_tagger.py`

- Examples are cut into windows of at most 128 tokens; batches of 32 are padded to equal length; padding gets label −100 so the loss ignores it.
- Loss: cross-entropy per token. Optimiser: AdamW, learning rate 5e-4, weight decay 0.01. Schedule: 5% warm-up then cosine decay. Gradient clipping at 1.0 prevents rare huge updates. Seeds fixed.
- Each epoch: train, then score span F1 on validation; save the checkpoint only when val F1 improves. The checkpoint stores the model config and which tokenizer it used.
- **Overfit sanity check**: before the real run, train on 10 examples for 150 epochs with no dropout. Loss must reach about 0; otherwise something is broken. This is how we know the layers are wired correctly.
- Real run: Google Colab GPU, 25 epochs, then the best checkpoint is scored on test, unseen templates and caps.

5.7 Regex baseline

Patterns for EMAIL, URL, IP, PAN (`AAAAA9999A`), AADHAAR (12 digits, optional grouping), DATE, ID_NUMBER (EMP/CASE/ACC/INV + digits) and PHONE (10–13 digits, checked by a counting function). Earlier patterns win overlaps. It cannot find names or places at all: that gap is a finding, not a bug.

5.8 Encoder-decoder comparison `seq2seq.py`

Same blocks as the tagger plus a decoder (2+2 layers, 1.69M parameters). It reads the sentence and *writes* the sanitised sentence token by token. Trained with teacher forcing, label smoothing 0.1, 15 epochs; output layer shares weights with the embedding (weight tying); greedy decoding. Target example: "Send the report to [PERSON] at [EMAIL]." Built to test the claim that generation is the wrong tool for redaction.

5.9 DistilBERT reference `train_distilbert.py`

The pretrained `distilbert-base-cased` with a token-classification head, fine-tuned 4 epochs on the same splits and scored with the same scorer. Never used in the product. It answers "how close does a from-scratch model get to a pretrained one, and at what size?".

5.10 Evaluation `metrics.py`

- **Exact-match span P/R/F1** (same start, end and label), per entity and overall.
- **Leaked spans**: share of true private spans not fully covered by predictions. The privacy metric. Ignores the label (hiding an email as [PHONE] still hides it).
- **Leaked characters**: catches half-hidden names.
- **Over-redaction**: harmless characters wrongly masked (usability cost).
- We wrote this ourselves, so no seqeval dependency.

5.11 Hybrid engine, API, ONNX

Covered in section 3. The first merge rule ("longest span wins, regex wins ties") scored *worse* than the tagger alone (F1 0.823) because loose regex overrode correct model labels (Aadhaar precision 0.40, phone 0.42). The priority rule fixed it. The ONNX export matches PyTorch to about 1e-5 (largest difference 9.7e-06 on the production model).

5.12 Browser extension `extension/`

- **Paste**: anything pasted into a text box on ChatGPT, Claude, Gemini, Copilot or Perplexity is cleaned first. Other sites: switch on "Protect this site" in the popup.
- **Shortcut Alt+Shift+P**: cleans the selection or the whole box on any page.
- **Replies**: placeholders on the page are displayed as the real values again (display only; the page's real text is unchanged).
- **Optional preview**: shows detections highlighted so the user can accept or reject each one before pasting.
- **Privacy**: only talks to 127.0.0.1:8000. Placeholder table lives in tab memory and is never sent anywhere. A bug fixed along the way: an endless loop where the page observer kept re-triggering itself.

5.13 PDF redaction `docs/pdf.py`

Reads each page's words with their box positions (PyMuPDF), runs the engine, converts spans to boxes, then a "document sweep" hides every other place the same value appears. Redaction is **true**: the text under the black box is deleted from the file, not just covered. It also removes links pointing at hidden text and wipes document metadata (author, title), then re-reads the finished file to list any detected word still visible. Not covered: scanned PDFs (no text layer) would need OCR.

6. The results, and how to read them

0.933
production hybrid F1 on test

2.8%
private spans left visible (test)

1.56M
parameters, 5.5 MB ONNX

All numbers are exact-match spans. "Leaked" = private info left visible (lower is better).

System	Params	F1 test	F1 unseen	Leaked test	Leaked unseen
Regex baseline	–	0.348	0.653	71.6%	51.5%
Encoder tagger, 2 layers	1.29M	0.855	0.718	5.7%	15.4%
Encoder tagger, 4 layers + case aug.	1.56M	0.921	0.787	3.7%	7.7%
Encoder-decoder (seq2seq)*	1.69M	exact 23%	exact 29%	1.6%†	2.2%†
DistilBERT (pretrained)	65.2M	0.956	0.942	2.3%	0.4%
Hybrid engine (production)	**1.56M**	**0.933**	**0.879**	**2.8%**	**5.4%**

* Scored on a different subset (rows up to 128 tokens), so do not compare it directly with the others. † Counts only verbatim copies of the full value; true leakage is higher (it produced things like `jagish [EMAIL]`).

Where each approach wins (F1 per entity, test)

Entity	Regex	Our tagger (2L)	DistilBERT
PERSON	0.00	0.83	0.94
LOCATION	0.00	0.89	0.96
EMAIL	0.98	0.87	0.97
PHONE	0.51	0.80	0.93
PAN, URL	1.00	1.00	1.00

Takeaway: regex wins on rigid formats, the model wins wherever meaning is needed, hence the hybrid.

Ablations (one change at a time, 15 epochs unless stated)

Variant	Params	F1 test	F1 unseen
Baseline: 2 layers, 4 heads, sinusoidal, 8k	1.29M	0.855	0.718
4 layers (25 epochs)	1.56M	0.910	0.809
6 layers (25 epochs)	1.82M	0.922	0.821
2 heads / 8 heads	1.29M	0.827 / 0.812	0.657 / 0.647
Learned positions	1.36M	0.865	0.618
Vocab 4k / 16k	0.78M / 2.32M	0.850 / 0.807	0.682 / 0.634

- **Depth is the biggest win.** 2 → 4 layers gives +5.5 F1 (equal 25-epoch budget). 4 → 6 gives only +1.2 for +17% parameters: diminishing returns. 4 chosen (smaller, same quality within noise, lowest leakage on unseen).
- **4 heads is best.** With d = 128, 8 heads leaves only 16 numbers per head.
- **Learned positions** match sinusoidal on test but collapse on unseen templates (0.618 vs 0.718): consistent with memorising template layouts while fixed waves generalise.
- **Vocabulary:** 4k equals 8k with 40% fewer parameters; 16k is worse because rare tokens are under-trained.
- **Caveat to state yourself:** single seed, and the unseen set is 595 rows from 5 templates. Trust trends (depth helps, 16k hurts), not decimals.

Case robustness result

On ALL-CAPS / lowercase text the old 2-layer tagger left 40.1% of private spans visible (PERSON recall 0.08). The retrained 4-layer, case-augmented model: 6.8% left visible, PERSON F1 0.12 → 0.78. Cost on normal test: none (F1 0.910 → 0.921).

Why the encoder-decoder lost (the evidence)

- Only **23%** of outputs exactly right; ordinary text changed in **35%** of rows; invented placeholders in 27%; dropped placeholders in 28%.
- Examples: "Episode Transcript Transcript" (duplicated word), "Geail" (corrupted word), `jagish [EMAIL]` (partly copied private text), an email labelled `[ID_NUMBER]`.
- On unseen sentence shapes it broke down completely. A tagger can only label, so it can never damage or invent text.

7. Why these design choices

Choice	Reason you can defend
PyTorch, not TensorFlow	Matched the plan, TensorFlow was only a release candidate for our Python 3.14, Hugging Face (needed for DistilBERT) is dropping TF support, and ONNX export is easier.
Encoder-only tagger as the main model	Redaction is "mark which words are private", not "rewrite the text". Labels cannot alter or invent text, and one forward pass is fast.
Own BPE	Learning requirement, and it lets us keep `@ . -` inside tokens so emails and IDs are not shredded. Offsets make character-exact redaction possible.
Small model (d=128, 4 layers)	Must run on the user's device. 5.5 MB runs anywhere; DistilBERT is 261 MB.
Hybrid, with a priority rule	Regex is perfect on rigid formats and the model is not; the model understands names and places and regex cannot. Priority rule came from an experiment that failed.
Recall over precision	Hiding one word too many is annoying; leaking a name is the failure the product exists to prevent. Word snapping and the loose-regex fallback both favour recall.
Everything local	If the text went to a cloud server for cleaning, the tool would itself leak. Server binds to 127.0.0.1 and only accepts the extension.
Unseen-templates split	Synthetic data is template-made; without this split we would be reporting memorisation.
Synthetic Faker data	No public dataset has Indian PAN or Aadhaar, and using real personal data would defeat the purpose.
Validator after redaction	A second safety net: re-scan the output; any regex hit blocks it.

8. Weak spots: say them before they do

Examiners respect honesty with numbers. Volunteering these makes you look in control.

1. Template familiarity inflates some scores

PAN, URL, AADHAAR and ORGANIZATION come only from template-generated data, so their 0.98–1.00 on `test` partly measures template familiarity. The honest generalisation number is the **unseen-templates**

score (hybrid F1 0.879, 5.4% leaked). The unseen set has no ORGANIZATION rows, so its ORGANIZATION 0.00 is pure false alarms, not a miss.

2. The from-scratch model trails the pretrained one

DistilBERT: F1 0.956 / 0.942 unseen, ours 0.921 / 0.787. Answer: it has 40× more parameters and large-scale pretraining. Ours is 5.5 MB and the hybrid closes much of the gap (0.879 on unseen).

3. Known failures of the live engine

Lowercase place names are missed ("in mumbai" stays visible). Common words are sometimes tagged: "Mail me at" → `[PERSON_1] me`; "His PAN" → `[LOCATION_1] PAN`. These over-redact (safe but ugly). "Infosys" was missed in one sentence. A 12-digit Aadhaar can be labelled `[ID_NUMBER_1]` instead of AADHAAR (hidden either way). Lowercase PAN is missed by regex (case-sensitive pattern).

4. Statistical limits

One training seed per ablation; unseen set only 595 rows from 5 templates. Validation F1 plateaus near 0.87 from epoch ~20 while training loss still falls (mild overfitting).

5. Things the checklist shows as not done

CRF layer (skipped), `run_benchmark.py` unified table script, latency benchmark, attention-heat-map notebook, data-exploration notebook, README. Do not quote a latency figure: we have none. Our numbers come from `reports/results.md`.

6. Scope limits

Mostly English plus Indian identifiers. Scanned PDFs need OCR. The extension needs the local server running (the in-browser ONNX route is exported but the extension uses the server). The engine still lacks Unicode/whitespace normalisation.

The honest one-liner

"It is a research prototype: strong on structured data and common names, weaker on lowercase or unusual names, and we report exactly where." Never say it is 100% safe.

9. Likely viva questions with answers

Basics

Q1 What problem does this solve?

Users paste private text into AI tools. PrivyLens replaces names, emails, IDs and so on with placeholders locally before sharing, and can restore them in the reply.

Q2 What is ATML / NLP in your project?

ATML: we built Transformer layers (attention, encoder, decoder), trained them, ran ablations and compared against a pretrained model. NLP: tokenisation (BPE), sequence labelling with BIO tags, span-level evaluation.

Q3 What do you mean by "from scratch"?

No pretrained weights and no library Transformer. Attention, positional encoding, encoder and decoder blocks are in `layers.py`; the tokenizer is our own BPE. We use PyTorch only for primitives (Linear, LayerNorm, optimiser). The only pretrained thing is DistilBERT, used purely as a reference and never in the product.

Q4 Why a Transformer rather than an LSTM or just regex?

Regex cannot recognise names (F1 0.00) because there is no pattern for "a person". Attention lets each word use context from the whole sentence ("send it to ___ at"), and processes all words in parallel.

The model

Q5 Explain self-attention simply.

Each word asks a question (query), every word offers a label (key) and content (value). The match score between query and keys, scaled by √(head size) and passed through softmax, becomes weights; the word's new vector is the weighted mix of values. So "Sharma" can pull information from "Rahul" and "to".

Q6 Why divide by $\sqrt{d_k}$?

Dot products grow with vector size, pushing softmax into saturated regions where gradients vanish. Dividing keeps scores in a healthy range.

Q7 Why multiple heads?

Each head can learn a different relationship (neighbouring word, nearby title like "Mr.", the preposition before a name). Ablation: 4 heads beat 2 and 8 at d=128.

Q8 Why positional encoding?

Attention treats the input as an unordered set; "Rahul Sharma" and "Sharma Rahul" would look the same. We add sin/cos waves to inject order. Learned positions did worse on unseen templates (0.618 vs 0.718).

Q9 What are residual connections and LayerNorm for?

Residual: keeps the original information and gives gradients a short path, so deeper stacks train. LayerNorm: keeps numbers in a stable range.

Q10 Why did you choose 4 layers?

Ablation. 2→4 layers: +5.5 F1 and leakage on unseen 15.4% → 7.0%. 4→6: +1.2 F1 for 17% more parameters and slightly worse unseen leakage. 4 is smaller and as good.

Q11 What do the 1.56M parameters consist of?

1.02M in the embedding table (8,000 × 128), about 0.53M in 4 encoder blocks, 3k in the output layer.

Q12 What is the difference between your encoder model and the encoder-decoder?

The encoder reads everything and labels each token in one pass. The encoder-decoder also has a decoder that generates the whole sanitised text token by token, using masked self-attention (no peeking ahead) and cross-attention to the input.

Q13 What is teacher forcing?

During training the decoder gets the true previous tokens as input, not its own predictions, so it learns faster and more stably. At test time it must use its own (greedy decoding), which is where errors snowball.

Data and tokenizer

Q14 Where did the data come from?

ai4privacy PII dataset (English, academic licence, never redistributed) mapped to our 11 labels, plus 5,000 synthetic sentences with Faker en_IN fake values. About 55k rows in total across splits.

Q15 How do you know labels on synthetic data are correct?

The generator inserts each value itself and records the exact character positions, so labels are correct by construction.

Q16 Explain BPE. Why not whole words or characters?

Whole words: unknown names and IDs become <UNK> and the vocabulary explodes. Characters: sequences get very long. BPE sits between: frequent chunks become single tokens, rare strings fall back to smaller pieces, so nothing is ever unknown.

Q17 How do predictions on tokens become redactions on text?

Each token stores its character offsets. The model labels tokens; `to_spans` merges B/I tags into spans with the first token's start and last token's end. Then redaction replaces those exact characters.

Q18 What is the unseen-templates test?

Some sentence templates are removed from training completely. Scores there show whether the model understands names in new contexts, rather than memorising familiar sentence frames.

Q19 How did you handle ALL-CAPS names?

A user found one undetected. Diagnosis: only 41 of 17,772 PERSON training examples were upper case. We augmented train with random upper/lower copies, retrained tokenizer and model, and added a `test_caps` set. Left-visible dropped from 40.1% to 6.8%. We also added word snapping.

Training and evaluation

Q20 How did you train and on what hardware?

Code written and unit-tested locally on CPU, then trained on Google Colab GPU. AdamW, lr 5e-4, 5% warm-up and cosine decay, batch 32, gradient clipping 1.0, 25 epochs, best checkpoint chosen on validation F1.

Q21 How did you know the model code was right?

Unit tests for attention and masks, BPE round trip, alignment recovery, plus the overfit test: 10 examples must reach loss ≈ 0. Also, we caught and fixed an embedding-initialisation bug (first run F1 0.21).

Q22 Why F1 and "leaked spans"? Which matters more?

F1 balances precision and recall. For privacy recall is primary because a miss exposes data; so we also report leaked spans, leaked characters (partial leaks) and over-redaction (usability cost).

Q23 Is your model overfitting?

Mildly. Validation F1 plateaus near 0.87 from epoch ~20 while training loss still falls. We keep the best-val checkpoint, use dropout 0.1 and weight decay. The unseen-template gap shows the remaining generalisation limit.

Q24 Why does DistilBERT win, and why not just ship it?

It is pretrained on large text so it already knows language. But it is 65.2M parameters (~261 MB) versus our 1.56M (5.5 MB). Our goal was a small local tool and a from-scratch learning project; the hybrid gets F1 0.933 on test.

Q25 Why did the seq2seq model fail?

Generation can change anything. It duplicated, corrupted or dropped words in 35% of test rows and invented placeholders. For redaction we need the untouched text preserved, which labelling guarantees by design.

System and privacy

Q26 How does merging regex and the model work?

Priority: precise regex (email, URL, IP, PAN) > model > loose regex (phone, Aadhaar, date, ID). The first simpler rule scored worse than the model alone, so we changed it based on measurements.

Q27 Is the data really not sent anywhere?

The server binds to 127.0.0.1, the extension talks only to that address, and the vault is in memory. Only the sanitised text goes to the AI site. Honest limit: the AI site still sees placeholders and surrounding context.

Q28 What if the model misses something?

A validator re-scans output with regex and blocks structured leaks; low-confidence spans (<0.8) are flagged; the preview lets the user review. Names the model misses can still leak, which is why we publish the 2.8% / 5.4% figures.

Q29 How does restore work?

Vault stores placeholder → value per session. `/restore` replaces `[PERSON_1]` in the reply with the original. The same value always gets the same number so replies stay consistent.

Q30 What is ONNX and why export?

A portable format that runs without PyTorch, even in a browser. Our tagger exports to 5.5 MB with outputs matching PyTorch to ~1e-5. It enables a future fully in-browser mode.

Q31 How is PDF redaction safe?

Boxes use real redaction: underlying text is deleted, linked URLs and metadata are removed, a sweep hides repeated values, and a final re-read checks for survivors.

Trick questions

Q32 "Your model is only 0.79 on unseen templates. Isn't that bad?"

That is the tagger alone, on purposely hard, unseen sentence shapes with a tiny set (595 rows, 5 templates). The production hybrid reaches 0.879, and DistilBERT with 40× the size is 0.942. We reported it because it is the honest generalisation number.

Q33 "Did you tune on the test set?"

No. Checkpoints were chosen on validation (first 1,000 rows); test is scored once at the end of a run. Tokenizer trained on the train split only.

Q34 "Why not use an existing library like Presidio or spaCy?"

Those are good products, but the project objective was to build and understand the Transformer and tokenizer ourselves and compare them against a pretrained reference. Our aim is the learning and the comparison, plus a small local engine.

Q35 "Can you prove it never leaks?"

No, and we do not claim that. End-to-end check on 300 test rows: 25 of 1,304 gold values (1.9%) visible verbatim. We state the leak rates per set and list known failures.

Q36 "What would you do next?"

Add a CRF layer for valid tag sequences, train longer and with more real-world names and lowercase text, add OCR for scanned PDFs, run in-browser with ONNX, add a latency benchmark and attention visualisations, and run several seeds for the ablations.

10. Live demo script

Setup (do this before the viva, not during)

1. Open a terminal in the project folder and start the engine: `python -m uvicorn api.server:app --host 127.0.0.1 --port 8000`
2. Chrome → `chrome://extensions` → turn on Developer mode → Load unpacked → pick the `extension` folder.
3. Confirm the server answers, then try one paste on claude.ai or chatgpt.com.

Show, in this order

Step	Do	Say
1	Paste: `Send the report to Rahul Sharma at rahul@example.com. His employee ID is EMP4821.` into a chat box.	"Name, email and ID are hidden before the site receives anything."
2	Type `MANAN MIHIR RAYCHAND DEDHIA` in a box and press Alt+Shift+P.	"This ALL-CAPS name failed originally. Case augmentation fixed it."
3	Show a reply containing `[PERSON_1]` being displayed as the real name.	"The vault is only in this tab's memory."
4	Run `python -m privylens.docs.pdf`, open a file from `pdf_out/`.	"Real redaction: text under the box is removed, metadata wiped."
5	Open `reports/results.md` and `reports/figures/tagger_architecture.png`.	"Every number in this viva comes from this file."

Other commands: `python -m pytest tests` (all tests); `python -m privylens.engine.pipeline` (re-scores the hybrid); `python -m privylens.evaluation.model_summary checkpoints/tagger.pt` (architecture figure and parameter table).

> **Known demo risks**
> Avoid sentences with lowercase place names ("in mumbai"). Do not demo "Mail me at ..." as a clean example: the model may tag "me". If the examiner types something odd and it fails, say: "That's the lowercase / unusual-name weak spot we list in the report", then show the validator and preview as the safety net.

11. Numbers to memorise

Fact	Value
Entity types / BIO tags	11 / 23

Training rows / val / test / unseen / caps	39,806 / 3,956 / 3,993 / 595 / 1,997
Synthetic rows	5,000
Tokenizer vocabulary	8,000 (BPE, own)
Model: d / heads / feed-forward / layers	128 / 4 / 256 / 4
Parameters: ours / seq2seq / DistilBERT	1.56M / 1.69M / 65.2M
Model files	tagger.pt 6.3 MB · ONNX 5.5 MB · DistilBERT ~261 MB
Training	AdamW · lr 5e-4 · batch 32 · 25 epochs · Colab GPU
Production hybrid F1 test / unseen / caps	0.933 / 0.879 / 0.906
Production hybrid leaked test / unseen / caps	2.8% / 5.4% / 4.9%
Regex F1 / leaked (test)	0.348 / 71.6%
DistilBERT F1 test / unseen	0.956 / 0.942
Seq2seq: exact match / text changed	23% / 35%
Depth gain 2 → 4 layers	+5.5 F1
Caps fix: left visible	40.1% → 6.8%
Code size · tests · commits	≈2,000 lines · 8 test files · 18 commits
Local server · shortcut	127.0.0.1:8000 · Alt+Shift+P

Final tip

If you do not know an answer, give the principle ("it is the privacy-versus-usability trade-off, so we favour recall"), then point to a number in results.md. Never invent a figure.